

AI Ride

Intelligent route overlap and trust-based matching
for decentralised cost-sharing mobility

System documentation — working flow, credential rules and deployment

Document	Contents
Section 1	What the system does
Section 2	Account and login credential rules
Section 3	Complete working flow, end to end
Section 4	How ride matching decides a result
Section 5	Trust score and ratings
Section 6	Screens and API reference
Section 7	Free hosting on Vercel and Render
Section 8	Running it locally
Section 9	Known limitations

Generated 24 July 2026

1. What the system does

AI Ride connects people who are **already travelling** with people who need to get somewhere along the same road. Nobody drives a dedicated trip. A rider publishes the journey they were making anyway; passengers whose pickup and drop both fall on that road can book a seat on it.

The two portals

Portal	Who uses it	What they do
Rider	Anyone already driving somewhere	Publish current location, destination, vehicle and free seats. Appear live on the map. Accept seat requests, start the trip with the passenger's code, end it at the drop.
Passenger	Anyone needing to travel	Enter pickup and drop. See only riders whose route covers both points, ranked by walking distance and overlap. Book, track live, pay, rate.

One account works for both roles — the same profile, ride ID and trust score follow you whichever side of the trip you are on.

Technology

Layer	Built with
Frontend	Vanilla JavaScript modules, hash router, no build step. Leaflet for maps.
Backend	FastAPI (Python), psycpg2
Database	PostgreSQL
Maps and routing	OpenStreetMap tiles, Nominatim geocoding, OSRM road routing
Messaging	Twilio SMS for phone OTP (optional — demo mode without it)

2. Account and login credential rules

2.1 Sign-up fields and validation

Field	Required	Rule enforced	Rejected example
Full name	Yes	At least 2 characters	P
Email	Yes	Valid address with a real, resolvable domain. Reserved and special-use domains are refused by the validator.	me@airide.local
Phone	Yes	At least 8 characters. Use international format for SMS to work.	99999
Government ID	Yes	At least 4 characters. Aadhaar, driving licence or passport number. Must be unique across accounts.	A1
Password	Yes	At least 6 characters	12345
Phone verification	No	Optional at sign-up. Completing it sets the verified flag, which adds +20 to the trust score.	—

2.2 How a password is stored

Passwords are never stored in readable form. On sign-up a 16-byte random salt is generated, and the database keeps `salt:sha256(salt + password)`. On login the same salt is re-applied and the two hashes are compared. The stored hash is stripped from every API response before it leaves the server.

For a production system this should be upgraded to bcrypt or Argon2, which are deliberately slow and therefore far harder to brute-force than SHA-256.

2.3 Identifiers and codes

Identifier	Format	Purpose and lifetime
Public ride ID	AR-000042	Issued on sign-up, derived from the account number, zero-padded to six digits. Permanent and never reused. Shown to the other party on every booking.
Phone OTP	6 digits	Sent by SMS to verify a phone number at sign-up. Valid 5 minutes, single use. Without Twilio credentials the app runs in demo mode and shows the code on screen.
Trip start code	4 digits	Generated when a seat is booked. Visible only to the passenger; the rider's own API response has the field removed. The rider must type it correctly to start the trip, which proves the two people actually met.

2.4 Login rules

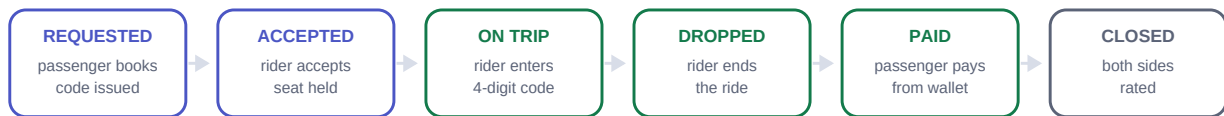
- Login is by **email and password**. The ride ID is an identifier, not a login credential.
- A wrong email returns *“No account found with this email.”* A wrong password returns *“Incorrect password.”*
- Signing in issues no token — see Section 9 on what that means.
- The session is held in browser storage; signing out clears it.

2.5 Demo accounts

Role	Email	Password	Ride ID
Passenger	praveen.test@example.com	demo1234	AR-000001
Rider	anusha@example.com	demo1234	AR-000002

These exist only in the local development database. A freshly deployed instance starts empty — create accounts through the sign-up screen.

3. Complete working flow



The six states a trip moves through. Every transition is recorded with a timestamp.

3.1 Step by step

#	Who	Action	What the system does
1	Rider	Opens the rider portal, sets current location and destination, picks bike, auto or car, free seats, cost share per km and departure time. Taps I'm available .	Requests the real driving route from OSRM and stores it. The rider appears on the live map as a vehicle marker with their route drawn.
2	Passenger	Opens the passenger portal, enters pickup and drop, seats needed and how far they will travel to meet the ride. Taps Find rides on my route .	Every live ride is scored against the trip. Only rides that pass both points, in the right order, within the radius are returned — ranked by walking distance, then overlap, then departure time.
3	Passenger	Reviews the results: fare, overlap percentage, walk to pickup, seats free, CO ₂ saved, and the rider's trust score. Taps Book seat .	Creates the booking at status requested , holds the seat, and issues a 4-digit start code shown only to the passenger.
4	Rider	Opens the request and taps Accept (or declines).	Status becomes accepted . Declining releases the seat back to the ride.
5	Rider	Drives to the pickup, asks the passenger for their code, types it in and taps Start ride .	The code is checked. A wrong code is refused and the trip stays put. A correct code sets status on trip and stamps the start time.
6	Both	The trip runs. The rider can push their live position.	The map redraws the vehicle against the route every few seconds for both parties.
7	Rider	Arrives at the drop and taps End ride .	Status becomes dropped with an end timestamp. Payment opens for the passenger.
8	Passenger	Taps Pay from wallet .	The fare moves from the passenger's wallet to the rider's in a single transaction. Insufficient balance is refused with a clear message. Status becomes paid .
9	Both	Each rates the other from 1 to 5 with an optional written note.	The score is averaged into that person's profile rating and their trust score is recalculated. Once both have rated, the trip closes.

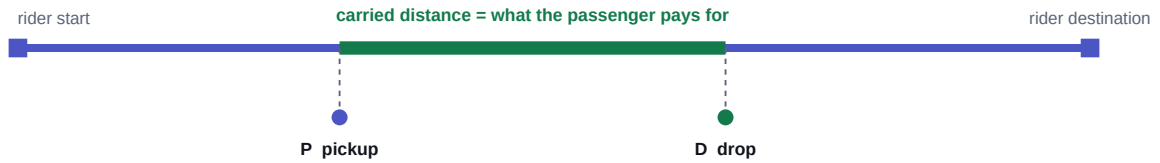
3.2 What cancels a trip

Who	When	Effect
Passenger	While requested or accepted	Booking cancelled, seat returned to the ride
Rider	While requested	Request declined, seat returned to the ride
Rider	Any time before starting	The whole ride can be cancelled, removing it from the map

A trip cannot be started without the correct code, cannot be paid before it is completed, and cannot be rated before it is finished. The server enforces each of these, not just the interface.

4. How ride matching decides a result

Rider's published road route (destination is irrelevant to the match)



Both ends must fall within the passenger's chosen walk radius, and the drop must come after the pickup along the route.

Only the stretch between P and D matters. The rider may continue far past the drop — that does not affect the match or the fare.

4.1 The rules, in order

#	Test	Fails when
1	The ride is live and has enough free seats	It is completed, cancelled, or fully booked
2	Pickup is close to the rider's route. Distance is measured to the route lines , not just its recorded points.	The pickup is beyond the chosen walk radius (1, 2, 5 or 10 km)
3	Drop is close to the rider's route	The drop is beyond the same radius
4	The drop comes after the pickup along the route	The passenger is travelling against the rider's direction

Nothing else disqualifies a ride. Overlap percentage is reported for ranking and display only.

4.2 Fare

fare = carried distance (km) × rider's rate per km × seats

Vehicle	Default rate	Typical seats
Bike	Rs.3.50 / km	1
Auto	Rs.4.50 / km	3
Car	Rs.6.00 / km	4

The rider can adjust their own rate when publishing. No commission is added and no surge is applied — the platform takes nothing, which is what keeps this a cost-sharing arrangement rather than a commercial taxi service.

5. Trust score and ratings

Every account carries a score from 0 to 100. It starts at 50 and moves with behaviour.

Factor	Effect
Verified identity	+20
Average rating	+3 per star (up to +15)
Completed trips	+1.5 each, capped at +15
Delivery success rate	up to +10
Cancellation rate	up to −25

Route deviations	-5 each, capped at -20
Reports filed against the user	-15 each, capped at -45
Slow response time	up to -10
Score	Risk band shown to the other party
90 – 100	Very low risk
75 – 89	Low risk
50 – 74	Medium risk
0 – 49	High risk

After a trip closes, each rating is averaged with every other score that person has received — as a rider and as a passenger — and the trust score is recomputed from the new average. Written notes appear on the profile.

6. Screens and API reference

6.1 Screens

Address	Screen	Sign-in needed
#/	Landing page with the live map	No
#/login, #/signup	Account access	No
#/home	Portal chooser, trust, wallet, recent activity	Yes
#/rider	Publish a ride, manage seat requests	Yes
#/passenger	Search and book rides	Yes
#/bookings	Your trips as a passenger	Yes
#/trips	Rides you have published	Yes
#/trip?id=N	Live trip tracking, start code, payment, rating	Yes
#/profile	Ride ID, trust breakdown, wallet	Yes

6.2 API

Method and path	Purpose
POST /users/register	Create an account, issues the ride ID
POST /auth/login	Sign in
POST /auth/send-otp	Send the 6-digit phone code
POST /auth/verify-otp	Check the phone code
POST /rides/publish	Rider goes available
GET /rides/live	Every ride currently on the map
POST /rides/search	Passenger match search
POST /rides/{id}/location	Push live position
POST /rides/{id}/status	available / started / completed / cancelled
POST /bookings	Book seats, issues the start code
GET /bookings/{id}	One trip, with live rider position
POST /bookings/{id}/status	accept / reject / cancel
POST /bookings/{id}/start	Verify the code and begin the trip
POST /bookings/{id}/complete	End the trip at the drop
POST /bookings/{id}/pay	Move the fare to the rider
POST /bookings/{id}/rate	Rate the other party
GET /users/{id}/reviews	Written reviews on a profile
GET /wallet/balance/{id}	Wallet and escrow balance

Interactive documentation is generated automatically at /docs on the running backend.

7. Free hosting on Vercel and Render

The frontend is static files, so it goes on Vercel. The backend needs Python and a database, so it goes on Render together with a free PostgreSQL instance. Both have free tiers that need no card.

7.1 Backend and database on Render

#	Step
1	Push the project to GitHub.
2	On Render choose New → Blueprint and select the repository. The included <code>render.yaml</code> creates the web service and the database together.
3	Wait for the first build. Render supplies <code>DATABASE_URL</code> automatically and the tables are created on first start.
4	Copy the service URL, for example <code>https://airide-api.onrender.com</code> . Open it — it should reply with a short JSON message.

7.2 Frontend on Vercel

#	Step
1	Edit <code>frontend/index.html</code> and put the Render URL into the backend location tag: <code><meta name="airide-api" content="https://airide-api.onrender.com"></code>
2	Commit and push.
3	On Vercel choose Add New → Project and import the repository. The included <code>vercel.json</code> already points at the frontend folder. Framework preset: Other . No build command is needed.
4	Deploy, then copy the resulting address, for example <code>https://ai-ride.vercel.app</code> .

7.3 Connect the two

In the Render service, set the environment variable `ALLOWED_ORIGINS` to the Vercel address and redeploy. Without this the browser blocks the requests. Vercel preview deployments on `*.vercel.app` are already permitted.

7.4 What the free tiers cost you

Limit	What it means in practice
The API sleeps after about 15 minutes idle	The first request afterwards takes roughly 50 seconds. Open the site a minute before a demonstration. The app shows a message explaining the wait rather than an error.
Free Postgres expires after 30 days	Export anything you need before then, or recreate the database. Not suitable for real users.
No background workers	Live positions update only while somebody has the page open.

8. Running it locally

Step	macOS and Linux	Windows (PowerShell)
Create the database	<code>createdb ride_sharing_db</code>	Use pgAdmin, or <code>createdb -U postgres ride_sharing_db</code>

Settings	<code>cp .env.example backend/.env</code> then set <code>DB_USER</code> to your macOS username and leave the password blank	Copy <code>.env.example</code> to <code>backend/.env</code> then set <code>DB_USER=postgres</code> and the password from the installer
Dependencies	<code>python3 -m venv .venv</code> <code>source .venv/bin/activate</code> <code>pip install -r requirements.txt</code>	<code>py -m venv .venv</code> <code>.\.venv\Scripts\Activate.ps1</code> <code>pip install -r requirements.txt</code>
Start the backend	<code>cd backend</code> <code>python -m uvicorn app:app --reload</code> <code>--port 8000</code>	Same command
Start the frontend	<code>cd frontend</code> <code>python3 -m http.server 8080</code>	<code>python -m http.server 8080</code>
Open	<code>http://127.0.0.1:8080/</code>	Same

Make sure nothing else is listening on port 8000. A stray file server there will answer instead of the API and every request will fail.

9. Known limitations

Stated plainly, because a marker will find them anyway and an honest list is worth more than a silent gap.

Limitation	Consequence	Fix
No session tokens	Requests carry a plain account number, so anyone who can reach the API can read or change another person's booking, including the start code.	Issue a signed token on login and check it on every request
Passwords hashed with salted SHA-256	Faster to brute-force than a purpose-built algorithm.	Move to bcrypt or Argon2
Phone codes held in memory	They are lost on restart and do not work across multiple server processes.	Store them in the database or a cache
Money stored as floating point	Rounding drift over many transactions.	Use a fixed-precision numeric column
Trust score is a fixed formula	It is weighted judgement, not a learned model — despite the project title.	Train a model on completed-trip outcomes
Positions update only on demand	Tracking is not continuous.	Add a background location stream

The first item is the one that matters before this is shown to anyone outside the team.